# SIMATS ENGINEERING
# CO2-ASSESSMENT TOOL-2

**COURSE NAME: Theory of Computation**

**COURSE CODE: CSA1304**

**COURSE OUTCOME COVERED**

**CO2: Design Finite Automata DFA, NFA, ε-NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)**

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5 Total Marks:25 Duration : 1 Hour**

# Design Task

***Code of Conduct:***

*I GIRIJA B/ 192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary*

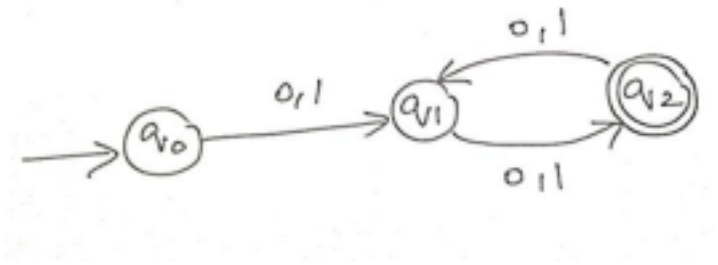
*action.*

***Signature of the student: GIRIJA B***

## SIMATS ENGINEERING

# ASSESSMENT TOOL

**1.Design the regular expression for the language represented by the given**




**automata**

From the given automaton:

- **Start state:** $q_0$
- $q_0 \xrightarrow{0,1} q_1$
- $q_1 \xrightarrow{0,1} q_2$(Final state)
- $q_2 \xrightarrow{0,1} q_1$

**Step 1: Analyze the Language**

- The first transition from $q_0$to $q_1$consumes **one symbol (0 or 1)**.
- The next transition from $q_1$to $q_2$consumes **another symbol (0 or 1)**.
- After reaching the final state $q_2$, every additional **two symbols** move:
  - $q_2 \rightarrow q_1 \rightarrow q_2$

Hence, the automaton accepts **all binary strings of even length (minimum length 2).**

Examples accepted:

- 00
- 01
- 10
- 11
- 0011
- 1010
- 110011

Examples rejected:

- ε
- 0
- 1
- 010
- 111

**Regular Expression**

$$\boxed{((0+1)(0+1))((0+1)(0+1))^*}$$

or equivalently,

$$\boxed{((0+1)(0+1))^+}$$

where **+** denotes **one or more occurrences**.

**2.Consider the following grammar**

**S->aSc|T|U|e**

**T-> bTc|e**

**U->aUb|e**

**A} What is the language defined by the grammar?**

**B)Give the derivation tree for the word aaabbc**

**C)A Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.**

**A) What is the language defined by the grammar?**

The grammar generates the union of three types of strings:

1. **Using** $S \to aSc$
    - Produces equal numbers of **a's** and **c's** surrounding strings generated by $S$.
2. **Using** $T \to bTc$
    - Produces strings of the form

$$b^n c^n,\ n \geq 0$$

3. **Using** $U \to aUb$
    - Produces strings of the form

$$a^n b^n,\ n \geq 0$$

Hence the language generated is

$$\boxed{L = \{a^i, b^j c^{i+j} \mid ij \geq 0\} \cup \{a^n b^n \mid n \geq 0\}}$$

That is,

- equal numbers of **a's** and **c's** with optional **b's** in the middle, and
- equal numbers of **a's** and **b's**.

**B) Derivation Tree for the word aaabbc**

The string is

$$aaabbc = a^3b^2c^1$$

It can be derived as follows:

$$\begin{aligned} S &\Rightarrow aSc \\ &\Rightarrow aaScc \\ &\Rightarrow aaTc \\ &\Rightarrow aabTcc \\ &\Rightarrow aaabbTccc \\ &\Rightarrow aaabbc \end{aligned}$$

**Parse Tree**


Leaves (left to right):

$$\boxed{aaabbc}$$

**C) Is the grammar ambiguous? Justify.**

A grammar is **ambiguous** if a string has **two or more different parse trees** or **two distinct leftmost/rightmost derivations**.

In this grammar,

- $S$can derive **ε** directly.
- $S$can also derive **ε** through

$$S \Rightarrow T \Rightarrow \varepsilon$$

And

$$S \Rightarrow U \Rightarrow \varepsilon$$

Thus, the empty string **ε** has multiple derivations.

**Example**

**First derivation** $S \Rightarrow \varepsilon$

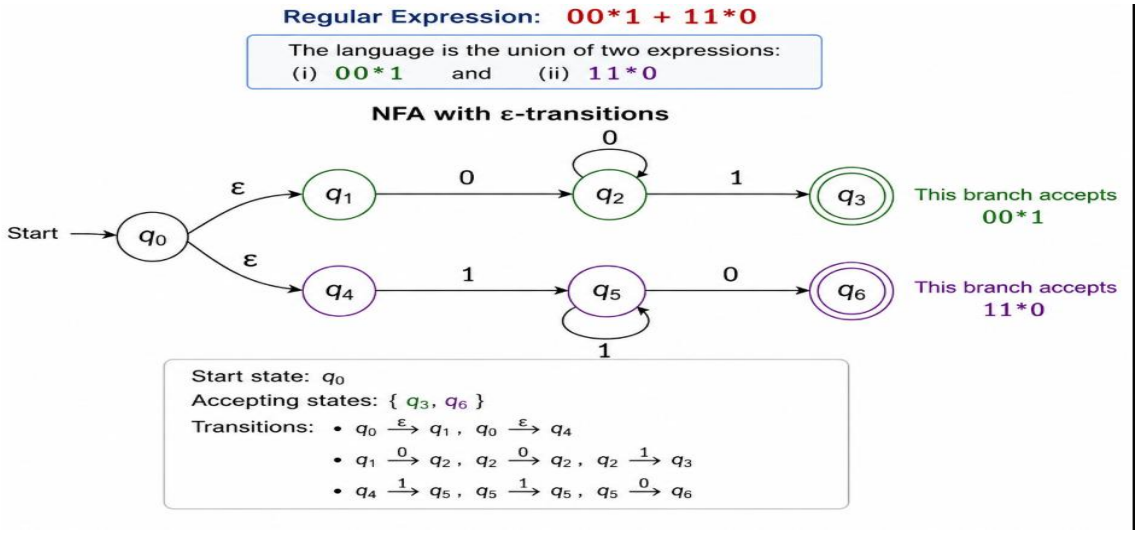
**Second derivation** $S \Rightarrow T \Rightarrow \varepsilon$

**Third derivation** $S \Rightarrow U \Rightarrow \varepsilon$

Since the same string (**ε**) has more than one derivation,

The grammar is ambiguous.

**3. Design a non deterministic finite automata with ε-transitions to represent the language denoted by the regular expression**
**00*1+11*0**




**4. I)Show that if L1 and L2 are regular then (L1+l2) also regular**

**Ii)Is the language { a^p| p is prime} regular? Justify**

(i) Show that if $L_1$and $L_2$are regular, then $(L_1 + L_2)$is also regular

Proof:

Let $L_1$and $L_2$be regular languages.

Since they are regular, there exist finite automata:

- $M_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$accepting $L_1$
- $M_2 = (Q_2, \Sigma, \delta_2, q_2, F_2)$accepting $L_2$

Construct a new NFA $M$:

- Add a new start state $q_0$.
- From $q_0$, add ε-transitions to the start states $q_1$and $q_2$.
- The accepting states are $F_1 \cup F_2$.

If the input is accepted by either $M_1$or $M_2$, then it is accepted by $M$.

Hence,

$$L(M) = L_1 \cup L_2 = L_1 + L_2$$

Since an NFA with ε-transitions is equivalent to a DFA, the resulting language is regular.

Conclusion:
The class of regular languages is closed under union. Therefore,

$$\boxed{L_1 + L_2 \text{ is regular.}}$$

---

4(ii) Is the language $\{a^p \mid p \text{ is prime}\}$regular? Justify. *(5 Marks)*

Answer: No. The language

$$L = \{a^p \mid p \text{ is prime}\}$$

is not regular.

Proof using the Pumping Lemma:

Assume $L$is regular.

Then there exists a pumping length $n$.

Choose the string

$$w = a^p,$$

where $p$is a prime number and $p > n$.

According to the Pumping Lemma,

$$w = xyz$$

such that:

- $| xy | \leq n$
- $| y | > 0$

Since the string contains only $a$'s,

$$y = a^k,\ k > 0.$$

Pump $y$twice ($i = 2$):

$$xy^2z = a^{p+k}.$$

Choose $p$such that $k$divides $p\ -$ ?More directly, pump with $i = p + 1$:

$$| xy^{p+1}z | = p + pk = p(1 + k),$$

which is composite because $p > 1$and $1 + k > 1$.

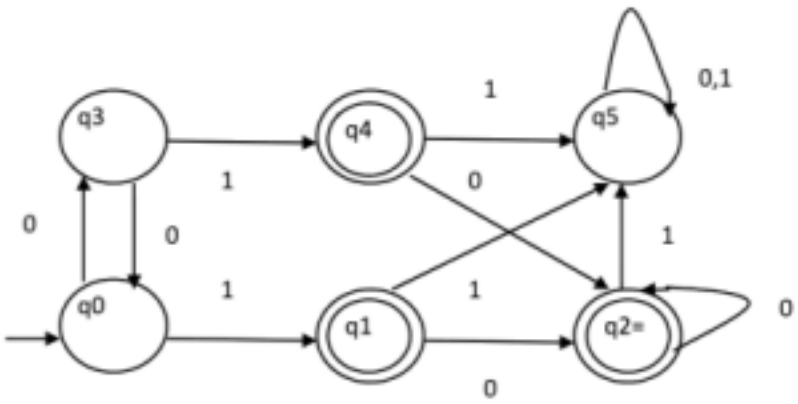
Thus the pumped string has a non-prime length, so it is not in $L$.

This contradicts the Pumping Lemma.

Conclusion:

$\{a^p \mid p$ is prime$\}$ is not a regular language.

**5. Design minimized automata, Consider the DFA given below. Find the states that are equivalent and construct a minimum state automata equivalent to the given automata. Draw the minimized automata, write its transition table and mark initial and final states**



## 5. Minimized DFA for the Given DFA

### (A) Equivalent States

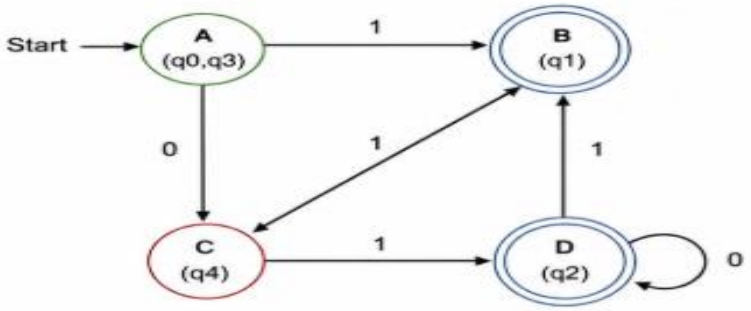
From partition refinement, the equivalent states are:

$$q_0 \equiv q_3$$

Other states are not equivalent to any states.

- A $(q_0,q_3)$ : Start state
- B $(q_1)$, D $(q_2)$ : Final states
- C $(q_4)$ : Non-final state

### (B) Minimized DFA (Diagram)




### (C) Transition Table of Minimized DFA

| Present State (Equivalent States) | Next State | | Final State? |
|---|---|---|---|
| | Input 0 | Input 1 | |
| A $(q_0,q_3)$ | C $(q_4)$ | B $(q_1)$ | No |
| B $(q_1)$ | D $(q_2)$ | C $(q_4)$ | Yes |
| C $(q_4)$ | D $(q_2)$ | D $(q_2)$ | No |
| D $(q_2)$ | D $(q_2)$ | B $(q_1)$ | Yes |

### (D) Summary

- Equivalent states: $q_0 \equiv q_3$
- Minimized DFA has **4** states: A $(q_0,q_3)$, B$(q_1)$, C$(q_4)$, D$(q_2)$
- Initial state: A $(q_0,q_3)$
- Final states: B$(q_1)$, D $(q_2)$

**Note:**
States $q_0$ and $q_3$ are equivalent because for every input string from either of these states, the DFA either accepts or rejects in the same way.